Modbus RTU Driver Library for Coolmay FX3G PLC

Terminology

- Register - an area of the memory in Modbus address stack.

Requirements

This library requires the following libraries to be installed:

- UtilsV4.sul
- TimeControV2.sul

> [!IMPORTANT]
>
> - This library requires the TCO Ticker to be set up in the Time Control 50 library. Configure it first.
> - Works only with GXW2 v1.91. Please install updates from `coolmay/soft` folder.

Changelog

v1.5 [09.04.25]

- add - `xWriteOnce` property for channel

v1.4 [09.04.25]

- add - clean all registers that are going to be used for data storage before the first read cycle.
- add - `xDone` property for channels. Produces a pulse when a channel completes one cycle.

v1.3 [28.02.25]

- fix - bug not restoring connection after timeout. Thanks @Alex_315.

v1.2 [12.12.24]

- change - Change `iReg` property of the channel to `Word[Unsigned]/Bit String[16-bit]`. Possible to set register address bigger 32000.
- add - Constants `MB_PORT_2`, `MB_PORT_3`, `MB_PORT_CAN` and `MB_PORT_TCP` for `iPort` channel property.

v1.1 [29.09.24]

- change `R` registers to `D` because some panels do not support `R` over Modbus protocol such as OP320.

v1.0 [22.09.24]

- Initial library with all features planned.

Description

This library enables configuration of a Coolmay FX3G PLC as a Modbus Slave or Modbus Master (on ports 2 and 3) for reading and writing data to Modbus devices over RTU. It provides a low-overhead interface for master configuration.

Coolmay PLC/HMI combo units have two RS485 ports: port 2 is available on the terminal connector, and port 3 is available on the DB9 connector. L02 series PLCs also have two RS485 ports, both on terminal connectors.

MB_PORT_SETTINGS

This function creates a variable with the correct bits for port initialization as master or slave.

Variable	Scope	Type	Description
`Parity`	INPUT	(Word[Signed])	Use constants `MB_PARITY_NONE` or `MB_PARITY_ODD` or `MB_PARITY_EVEN`
`StopBit`	INPUT	(Word[Signed])	Use constants `MB_STOPBIT_1` or `MB_STOPBIT_2`
`Baudrate`	INPUT	(Word[Signed])	Use constants `MB_BPS_600` or `MB_BPS_1200` or `MB_BPS_2400` or `MB_BPS_4800` or `MB_BPS_9600` or `MB_BPS_19200` or `MB_BPS_38400` or `MB_BPS_57600` or `MB_BPS_115200`

Declare a local Unsigned Double Word variable in your program, such as `PortSettings`, then call the function.

```
PortSettings := MB_PORT_SETTINGS(MB_PARITY_NONE, MB_STOPBIT_1, MB_BPS_9600);
```

Modbus Slave

MB_SLAVE_INIT_PORT2, MB_SLAVE_INIT_PORT3

This function initializes the Modbus slave on port 2 or port 3 of the PLC.

Variable	Scope	Type	Description
`xInit`	INPUT	BIT	Signal to init data. Initialization occurs on every rising edge of this parameter.
`iAddress`	INPUT	(Word[Signed])	What will be the address of this PLC in a Modbus network
`PortSettings`	INPUT	(Double word[Signed])	Result of MB_PORT_SETTINGS function

Example setup of slave with address 1 on port 2.

```
PortSettings := MB_PORT_SETTINGS(MB_PARITY_NONE, MB_STOPBIT_1, MB_BPS_9600);
M0 := MB_SLAVE_INIT_PORT2(TRUE, 1, PortSettings);
```

No further configuration is required to operate the PLC as a slave on port 2. The same approach applies to port 3. `M0` is not referenced elsewhere — it is a formal requirement of the function call syntax; the assignment must have a left-hand side.

Modbus Master

MB_MASTER_INIT_PORT2, MB_MASTER_INIT_PORT3

These functions initialize the Modbus Master on port 2 (terminal connector) or port 3 (DB9 connector) respectively. Both accept the same parameters.

Variable	Scope	Type	Description
`xInit`	INPUT	BIT	Signal to init data. Initialization occurs on every rising edge of this parameter.
`PortSettings`	INPUT	(Double word[Signed])	Result of MB_PORT_SETTINGS function

Example setup of master on port 2.

```
PortSettings := MB_PORT_SETTINGS(MB_PARITY_NONE, MB_STOPBIT_1, MB_BPS_9600);
M0 := MB_MASTER_INIT_PORT2(TRUE, PortSettings);
```

MB_PROCESS_50

This function block processes read and write operations for all configured channels.

> Before use, install the TimeControl50 library and configure `TCO_TICKER_50`. This ticker increments at 50 ms intervals.

Variable	Scope	Type	Description
`mb_xEnable`	INPUT	BIT	Start processing channels.
`mb_iBuffer`	INPUT	(Word[Signed])	Number of `D` device for registers and `M` device for coils to use as a buffer. For instance if 100, then `D100` will be used as first device for buffer or `M100` for coils. It will take as many devices as many registers or coils are read from a single channel (`iNum`).
`mb_Timeout`	OUTPUT	(Word[Signed])	Number of device that timed out.

The global array `MB_CHANNELS` of 30 `MB_REG_50` elements is declared in the library. No user declaration is required. This array defines the list of channels to process. Each channel may read or write up to 125 registers. The array must be configured once, typically on PLC startup using the `M8002` flag.

The following table lists the parameters of the `MB_REG_50` structure.

Variable	Type	Description
iDDevNum	(Word[Signed])	Number of D device to store result value for registers and M device for coils. For instance if set to K200 then result of read will be in D200 if read register or M200 if read coil.
iNum	(Word[Signed])	Number of registers or coils to read. By default 1.
iReg	Word[Unsigned]/Bit String[16-bit]	Start Modbus (holding\input) register address in decimal.
iRF	Word[Unsigned]/Bit String[16-bit]	Modbus read Function. Default is H3.
iWF	Word[Unsigned]/Bit String[16-bit]	Modbus write Function. Default is H6.
iDev	Word[Unsigned]/Bit String[16-bit]	Address of slave device.
tCycle	(Word[Signed])	Cycle interval for this channel, in 50 ms increments. For example, to read once every 100 ms, set tCycle to 2. Set to 0 for manual-only reads via the xReadOnce parameter.
iWR	(Word[Signed])	Read write mode. Default is MB_READ_WRITE. MB_READ or MB_WRITE could be set.
iPort	(Word[Signed])	Communication port. Port 2 — 0, Port 3 — 1, CAN — 2, Ethernet Modbus TCP — 3.
xDone	Bit	Channel read/write cycle completed. Primarily intended for xReadOnce channels with manual cycles.
xEnabled	Bit	The channel is processed only when enabled.
xReadOnce	Bit	On a rising edge (OFF → ON), triggers a single read of this channel. For manual-only reads, set tCycle to 0.
xWriteOnce	Bit	On a rising edge (OFF → ON), triggers a single write of this channel. For manual-only writes, set tCycle to 0.
xWriteOnChange	Bit	When TRUE, changes are written to the slave immediately upon detection. When FALSE, changes are written only when the tCycle interval elapses.

Supported functions

- H1 - Read Coils
- H2 - Read Discrete Inputs
- H3 - Read Holding Register
- H4 - Read Input Register
- H5 - Write Single Coil
- H6 - Write Single Register
- HF - Write Multiple Coils

- `H10` - Write Multiple Registers

Supported ports

- `MB_PORT_2` - RS485 first port A,B
- `MB_PORT_3` - RS485 second port A1,B1
- `MB_PORT_CAN` - CAN port H,L
- `MB_PORT_TCP` - Ethernet port

`iDDevNum`

Each channel reserves twice the number of devices specified by `iNum`. If `iNum` is `1` and `iDDevNum` is `200`, `D200` holds the result and `D201` is used internally. If `iNum` is 5, `D200`–`D204` hold results and `D205`–`D209` are used for internal calculations. This must be taken into account when configuring channels.

For instance, consider the following configuration:

```
MB_CHANNELS[0].iDDevNum := 600;
MB_CHANNELS[0].iNum := 3;

MB_CHANNELS[1].iDDevNum := 603;
MB_CHANNELS[1].iNum := 1;
```

This configuration appears correct at first glance because the second channel uses the next free device number, but it does not account for the internal calculation devices and will therefore fail.

When reading 3 registers into `D600`, the next available device is `D606`.

`iWF`

Two functions are supported for writing registers: `H6` writes a single register, and `H10` writes a group of registers. When `H6` is set on a multiple-register channel, only the changed register is written individually upon detection. When `H10` is set on a multiple-register channel, a change to any register in the group triggers a single request that updates all registers in that group. For double-word values, use `H10`. If the channel contains multiple independent single-register variables, use `H6`. For a single-register channel, use `H6` as well.

The same rule applies when working with coils. Use `H5` even on multiple-coil channels.

Example

Example program.

```
IF M8002 THEN

    (* Number of consecutive timeouts before a channel is marked as suspended.
Default: 2 *)
    MB_TIMEOUT_COUNT := 2;
    (* How often to retry suspended channel in 50ms increments: Default 80 (4
seconds) *)
```

```
    MB_SUSPEND_RETRY := 80;
    (* How long no response is a timeout: Default 4 (200 milliseconds) *)
    MB_TIMEOUT_TIME := 4;

    PortSettings := MB_PORT_SETTINGS(MB_PARITY_NONE, MB_STOPBIT_1, MB_BPS_9600);
    (* Multi master available for both ports for L02 *)
    M0 := MB_MASTER_INIT_PORT2(TRUE, PortSettings);
    M0 := MB_MASTER_INIT_PORT3(TRUE, PortSettings);


    MB_CHANNELS[0].xEnabled := TRUE;
    MB_CHANNELS[0].iDDevNum := 600;
    MB_CHANNELS[0].iNum := 3;
    MB_CHANNELS[0].iReg := K16384;
    MB_CHANNELS[0].iRF := H3;
    MB_CHANNELS[0].iWF := H6;
    MB_CHANNELS[0].iDev := H1;
    MB_CHANNELS[0].tCycle := 20; (* 20 * 50ms = 1000ms or 1 second *)
    MB_CHANNELS[0].xWriteOnChange := TRUE;
    MB_CHANNELS[0].iWR := MB_READ_WRITE;
    MB_CHANNELS[0].iPort := MB_PORT_2;


    (* Minimum setup and will connect devices through Port 3 *)
    MB_CHANNELS[1].xEnabled := TRUE;
    MB_CHANNELS[1].iDDevNum := 610;
    MB_CHANNELS[1].iReg := K16385;
    MB_CHANNELS[1].iDev := K16;
    MB_CHANNELS[1].tCycle := 4;
    MB_CHANNELS[1].iWR := MB_READ;
    MB_CHANNELS[1].iPort := MB_PORT_3;

    (* Manual cycle read and write of a group 3 registers *)
    MB_CHANNELS[2].xEnabled := TRUE;
    MB_CHANNELS[2].iNum := 3;
    MB_CHANNELS[2].iDev := 1;
    MB_CHANNELS[2].iPort := MB_PORT_3;
    MB_CHANNELS[2].iDDevNum := 10;
    MB_CHANNELS[2].iReg := H1000;
    MB_CHANNELS[2].iRF := H3;
    MB_CHANNELS[2].iWR := MB_READ;
    MB_CHANNELS[2].tCycle := 0;
END_IF;

fbMbProcess(mb_xEnable := TRUE, mb_iBuffer := 100);

(* manually read channel 2 once *)
IF M1 THEN
    MB_CHANNELS[2].xReadOnce := TRUE;
    IF MB_CHANNELS[2].xDone = TRUE THEN
        M1 := FALSE;
        MB_CHANNELS[2].xReadOnce := FALSE;
    END_IF;
END_IF;
```

```
(* manually write channel 2 once *)
IF M2 THEN
    D10 := 100;
    D11 := 100;
    D12 := 100;
    MB_CHANNELS[2].xWriteOnce := TRUE;
    IF MB_CHANNELS[2].xDone = TRUE THEN
        M2 := FALSE;
        MB_CHANNELS[2].xWriteOnce := FALSE;
    END_IF;
END_IF;

(* Another way to check if write was successful
when manually write channel 2 once *)
IF M2 THEN
    D10 := 200;
    D11 := 200;
    D12 := 200;
    MB_CHANNELS[2].xWriteOnce := TRUE;
    IF D10 = D13 AND D11 = D14 AND D12 = D15 THEN
        M2 := FALSE;
        MB_CHANNELS[2].xWriteOnce := FALSE;
    END_IF;
END_IF;
```

After configuration, `D600`, `D601`, and `D602` will hold the results of the first channel from device address 1, updated every second. If a value changes, the slave is updated immediately.

`D610` will hold the result of the second channel. Since this channel is read-only (`MB_READ`), any local change to `D610` will be overwritten by the next read from the slave in 200 ms.

Alternative Write Confirmation

The logic behind `IF D10 = D13 AND D11 = D14 AND D12 = D15 THEN` is as follows: as described in the `iDDevNum` section, each channel reserves twice the number of devices. Setting `iDDevNum` to 10 means the result is stored starting at `D10`. When reading one register, `D10` holds the actual value and `D11` serves as a change-tracking buffer. When reading 3 registers, `D10`–`D12` hold the actual values and `D13`–`D15` hold the change-tracking buffer, which is updated to the current values on a successful write. If the values match their corresponding buffers, the write was successful.

This is particularly relevant when using `xWriteOnChange` (rather than `xWriteOnce`) with a channel of 10 registers and write function `H6`. If 2 registers are changed, `H6` writes them one by one, setting `xDone` on each success. Counting 2 pulses on `xDone` can rapidly become complex. In contrast, a comparison such as `IF D10 = D13 AND D11 = D14 AND D12 = D15 THEN` confirms that all changed registers were successfully updated in a single check.

Timeouts

The library implements a channel suspension mechanism. If a channel times out for `MB_TIMEOUT_COUNT` consecutive attempts, it is marked as suspended. Suspended channels are requested once every

`MB_SUSPEND_RETRY` interval. As soon as a response is received, the suspended mark is removed and the channel resumes its normal cycle according to `MB_CHANNELS[*].tCycle`.